

怎么在「不是 Claude Code」的工具里用观澜 agent + skills

这套 agent + skill 不绑死 Claude Code。原因：

- 所有「能力」其实是 fof/ 下的 Python 引擎 + outputs/*.json 数据契约
- 「skill」是薄包装脚本 (.claude/skills/<name>/scripts/*.py)，可以脱离任何 AI 工具直接 python 跑
- 「agent」是一份工作流约束 —— 用任意 LLM 加载本仓库根的 AGENTS.md 当系统提示即可

下面分场景给「怎么装 / 怎么调用」的最小步骤。

两条通用行为（任何 AI 工具都一致，不只 Claude Code）

agent 有两条固定行为，不依赖某个工具：

1. 首轮自动给「速用指南」（这个 agent 做什么 / 5 个 skill 一句话 / 怎么触发 / 文档链接）；
2. 每条回复固定以「要打开仪表盘吗？」收尾。

它们分两层落地，确保跨工具一致：

工具类型	行为来自哪里
读系统提示的 LLM 工具（Cursor / Codex / Continue / Cline / Gemini / ChatGPT 粘贴 …）	写在仓库根 AGENTS.md 工作流（step 0 ONBOARD + step 6 固定收尾）
Claude Code	.claude/agents/guanlan-analyst.md（同源逻辑，含 frontmatter）
不读系统提示的工具 / 纯命令行 / CI	scripts/guanlan_brief.py 的输出本身自带：顶部「速用指南」 + 末尾「要打开仪表盘吗？」（--no-guide 可关顶部）

也就是说：哪怕某个工具完全不认 AGENTS.md，只要它能跑一句 python scripts/guanlan_brief.py

或贴它的输出，这两条行为照样出现 —— 真正与工具无关。

共同准备（一次性）

```
git clone <repo> && cd guanlan-regime-factor
python -m venv venv && source venv/bin/activate      # win: venv\Scripts\Activate.ps1
pip install -r requirements.txt
cp .env.example .env                                # ■ TUSHARE_TOKEN■■■■ LLM_* ■■■
```

仪表板看 demo（无任何 key 也行）：

```
python scripts/open_dashboard.py                    # http://127.0.0.1:8000
```

方案 A：完全不要 LLM —— 命令行直出研判简报

最普适，任何 Python 环境都能用：

```
python scripts/guanlan_brief.py # ■ outputs/*.json + vault/ ■ Markdown
python scripts/guanlan_brief.py --recompute --asof 2026-06-09 # ■■■ pipeline ■■■■
python scripts/guanlan_brief.py -o docs/latest-brief.md # ■■■
```

没有 AI，没有联网调用，逻辑全部确定性。集成进任何脚本/CI 都行。

输出已自带「速用指南」(顶) + 「要打开仪表盘吗？」(尾) 两条通用行为；CI/重复场景加 `--no-guide` 省掉顶部指南。

方案 B：用别的 AI 工具加载 AGENTS.md

Claude Code (项目级)

项目根已经有 `.claude/agents/guanlan-analyst.md`，开箱可用。说「跑一遍大势研判」即触发。

Cursor / Cursor Desktop

Cursor 自动读取仓库根的 AGENTS.md（新版）或 .cursorrules（旧版）。已就绪：

Cursor ██████████ AGENTS.md

旧 Cursor 用户可创建一个 thin pointer :

```
cp AGENTS.md .cursorrules # ■■■■■■
```

OpenAI Codex CLI / Codex (VS Code)

Codex 的官方约定就是 AGENTS.md (multi-file: 仓库根优先于全局)。已就绪, 无需额外配置。

GitHub Copilot Workspace / Custom Instructions

在 Copilot 设置里把 AGENTS.md 内容粘进「Custom instructions for this repository」。

Continue.dev (VS Code / JetBrains)

`.continue/config.json` 里加一个 `customCommands`, `systemMessage` 字段读 `AGENTS.md` :

```
// .continue/config.json
{
  "customCommands": [{
    "name": "guanlan",
    "prompt": "<paste AGENTS.md [redacted] [redacted] file:///path/AGENTS.md>",
    "description": "[redacted] . [redacted]"
  }]
}
```

Cline / Roo Code (VS Code)

设置面板的 System Prompt 直接粘 AGENTS.md 全文。模型用你已配的任何 provider。

Aider

```
aider --message-file AGENTS.md # ■■■■ AGENTS.md ■■■■■■  
# ■■■■  
aider --read AGENTS.md # ■ AGENTS.md ■■■■■■■■■■
```

Gemini CLI

新建 `GEMINI.md`（Gemini CLI 的约定文件），内容直接 `cp AGENTS.md GEMINI.md`：

```
cp AGENTS.md GEMINI.md
gemini # █ █ █ █ GEMINI.md
```

ChatGPT Web / Claude.ai / DeepSeek 网页版

1. 新建对话，系统提示框粘贴 AGENTS.md 全文
2. 用户消息：「跑一遍大势研判」/「现在该进攻还是防御」
3. 模型按 6 步工作流回答；模型不会自动跑 Python，所以你手动在本地跑这几条命令并把输出贴回去：

```
python scripts/run_pipeline.py --asof <YYYY-MM-DD> --start 2020-01-01
python .claude/skills/quant-research-retriever/scripts/query_vault.py "HMM regime" --top 5
python scripts/quanlan_brief.py
```

Ollama / 本地 LLM

ollama run llama3.2 < AGENTS.md # █ AGENTS.md ██████████

或在 OpenWebUI 设置 → System Prompt 粘贴。

方案 C：把仪表盘内嵌的 AI 顾问指向任意厂商

仪表板自带的「实时 AI 顾问」（右上角 AI 配置弹窗）已支持：

厂商	走的协议	怎么填
Anthropic	原生	LLM_PROVIDER=anthropic + ANTHROPIC_API_KEY
OpenAI	OpenAI	LLM_PROVIDER=openai + OPENAI_API_KEY
DeepSeek	OpenAI-兼容	LLM_PROVIDER=openai + LLM_API_KEY + LLM_BASE_URL=https://api.deepseek.com
Moonshot Kimi	OpenAI-兼容	base_url=https://api.moonshot.cn/v1
智谱 GLM	OpenAI-兼容	base_url=https://open.bigmodel.cn/api/paas/v4
通义千问	OpenAI-兼容	base_url=https://dashscope.aliyuncs.com/compatible-mode/v1
Ollama 本地	OpenAI-兼容	base_url=http://localhost:11434/v1

UI 上 7 个预设按钮, 1 click 填好 base_url + 推荐模型, 输入 key 即可。

数据契约（你能信赖的「公共表面」）

任何工具/任何 LLM 接入时，只依赖这 5 份 JSON 的字段名，不依赖任何工具特定 API：

文件	关键字段
outputs/regime.json	asof, composite_score, band, regime_label, equity_exposure, indicators[]
outputs/master.json	verdict, master_score, confidence, hmm.state_name, er.value, transition.matrix, state_stats[]
outputs/factors.json	ranking.factors[], rotation_ic.ic_mean/icir/hit_rate, tearsheet[]
outputs/factor_allocation.json	posture, tilt_mode, overweight[], underweight[], caveats[]
outputs/dashboard.json	上面四块的打包快照

字段稳定 → 任何工具读这个 JSON 都能渲染。

工具能力对照表

工具	加载 AGENTS.md	跑 Python skill	直接看仪表盘	调 AI 顾问
Claude Code	自动	Bash	preview	Anthropic/OpenAI-兼容任选
Cursor	自动	Terminal	浏览器	
Codex CLI	自动		—	—
Aider	--read	用户手动	浏览器	—
Gemini CLI	GEMINI.md	shell	浏览器	—
Ollama	粘提示	用户手动	浏览器	设 LLM_BASE_URL=http://localhost:11434/v1
ChatGPT Web	粘提示	用户手动	浏览器	通过 AI 配置
无 AI（纯 CLI）	—			离线基线建议

仅供量化研究参考，不构成投资建议。